AVA

Autonomous Voice Agent
Complete Technical Summary

Phases 1 – 4 · C# .NET 8 · Azure Cognitive Services · Claude LLM · EF Core + SQLite

AVA — Complete Technical Summary

Everything built so far, every file explained, every technical term defined.

What is AVA?

AVA (Autonomous Voice Agent) is a backend-first emotional support chatbot that:

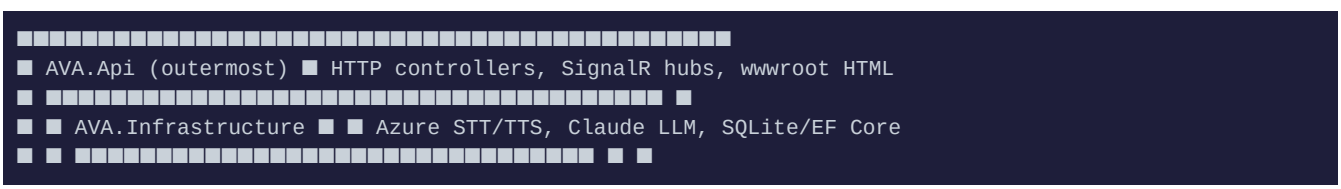
- Accepts a user's **typed or spoken message**
- Detects the user's **emotional state** from their words
- Sends the message + emotion context to **Claude (Anthropic's LLM)** for an intelligent response
- Converts Claude's text response to **speech audio** using Azure Neural TTS
- Plays the audio back to the user in the browser

The stack is **C# .NET 8** backend + **vanilla HTML/JS** frontend (no React, no Angular — just a browser page for testing).

Architecture: Clean Architecture

The project follows **Clean Architecture** (also called Onion Architecture). The rule is simple:

Inner layers know nothing about outer layers.



```
■ ■ ■ AVA.Application ■ ■ ■ Interfaces, DTOs, orchestration logic
■ ■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■ ■
■ ■ ■ ■ AVA.Domain ■ ■ ■ ■ Pure C# entities, enums — zero dependencies
■ ■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■ ■
■ ■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■
■ ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Why this matters:

- `AVA.Domain` has zero NuGet packages. It compiles in milliseconds and is never broken by framework updates.
- `AVA.Application` defines *what* services do (interfaces), but never *how* they do it.
- `AVA.Infrastructure` does the actual work (calls Azure, calls Claude, talks to SQLite).
- `AVA.Api` wires everything together and exposes HTTP endpoints.

Dependency Rule: `AVA.Api` → `AVA.Infrastructure` → `AVA.Application` → `AVA.Domain`. Arrows never point inward. Domain never imports Application. Application never imports Infrastructure.

Solution Structure

```
AVA/
■■■ AVA.sln ← Visual Studio solution file (links all projects)
■
■■■ src/
■ ■■■ AVA.Domain/ ← Pure business types, no dependencies
■ ■ ■■■ Entities/
■ ■ ■ ■■■ Session.cs ← Domain concept of a conversation session
■ ■ ■ ■■■ Message.cs ← Domain concept of one message
■ ■ ■ ■■■ UserProfile.cs ← Domain concept of a user
■ ■ ■■■ Enums/
■ ■ ■■■ EmotionType.cs ← Neutral, Sad, Anxious, Lonely, Angry, Happy, Motivated, Hopeful
■ ■ ■■■ MessageRole.cs ← User, Assistant
■ ■
■ ■■■ AVA.Application/ ← Business logic interfaces + DTOs
■ ■ ■■■ DTOs/
■ ■ ■ ■■■ ChatMessage.cs ← record(Role, Content) — sent to Claude
■ ■ ■ ■■■ ConversationContext.cs ← record(SessionId, Messages, UserName, UserGoals)
■ ■ ■■■ Interfaces/
■ ■ ■ ■■■ ISpeechToTextService.cs ← TranscribeAsync(byte[]) → string
■ ■ ■ ■■■ ITextToSpeechService.cs ← SynthesizeStreamAsync(text) → IAsyncEnumerable<byte[]>
■ ■ ■ ■■■ ILlmProvider.cs ← StreamCompletionAsync(messages, systemPrompt) →
IAsyncEnumerable<string>
■ ■ ■ ■■■ IEmotionAnalyzer.cs ← Analyze(text) → EmotionType
■ ■ ■ ■■■ IContextManager.cs ← GetContextAsync / SaveExchangeAsync / ClearSessionAsync
■ ■ ■ ■■■ ISystemPromptBuilder.cs ← Build(context, emotion) → string
■ ■ ■■■ Services/
■ ■ ■■■ ConversationOrchestrator.cs ← THE BRAIN — runs the full pipeline
■ ■ ■■■ DynamicSystemPromptBuilder.cs ← Builds Claude's system prompt per emotion
■ ■ ■■■ InMemoryContextManager.cs ← Phase 3 in-memory store (replaced in Phase 4)
■ ■
■ ■■■ AVA.Infrastructure/ ← External integrations
■ ■ ■■■ Settings/
■ ■ ■ ■■■ AzureSettings.cs ← SpeechKey, SpeechRegion
■ ■ ■ ■■■ ClaudeSettings.cs ← ApiKey, Model, MaxTokens
■ ■ ■■■ Stt/
■ ■ ■ ■■■ AzureSpeechToTextService.cs ← Calls Azure Cognitive Services STT
■ ■ ■■■ Tts/
■ ■ ■ ■■■ AzureTextToSpeechService.cs ← Calls Azure Neural TTS (Jenny voice, WAV output)
■ ■ ■■■ Llm/
■ ■ ■ ■■■ AnthropicLlmProvider.cs ← Calls Claude via Anthropic.SDK, streams tokens
```

```
■ ■ ■■■ Emotion/
■ ■ ■ ■■■ RuleBasedEmotionClassifier.cs ← Keyword matching → EmotionType
■ ■ ■■■ Context/
■ ■ ■ ■■■ EfCoreContextManager.cs ← Phase 4 — reads/writes SQLite via EF Core
■ ■ ■■■ Persistence/
■ ■ ■■■ AvaDbContext.cs ← EF Core DbContext, schema configuration
■ ■ ■■■ ConversationSession.cs ← EF Core entity: one DB row per session
■ ■ ■■■ ConversationMessage.cs ← EF Core entity: one DB row per message
■ ■
■ ■■■ AVA.Api/ ← ASP.NET Core Web API
■ ■■■ Controllers/
■ ■ ■■■ VoiceTestController.cs ← HTTP endpoints: /greet /chat /transcribe /synthesize /ping
■ ■■■ Hubs/
■ ■ ■■■ VoiceHub.cs ← SignalR WebSocket hub (real-time voice streaming)
■ ■■■ wwwroot/
■ ■ ■■■ index.html ← Browser test page (hold-to-record mic + text input)
■ ■■■ appsettings.json ← Non-secret config (placeholder API keys)
■ ■■■ appsettings.Development.json ← REAL API keys — GITIGNORED, never committed
■ ■■■ Program.cs ← App startup, DI wiring, middleware pipeline
■
■■■ tests/
■■■ AVA.UnitTests/ ← L0 unit tests (xUnit + Moq + FluentAssertions)
■ ■■■ Api/
■ ■ ■■■ VoiceTestControllerTests.cs ← 10 tests for HTTP controller logic
■ ■■■ Context/
■ ■ ■■■ EfCoreContextManagerTests.cs ← 7 tests for database context manager
■ ■■■ Domain/
■ ■ ■■■ SessionTests.cs ← Tests for domain entity business rules
■ ■■■ Stt/
■ ■ ■■■ AzureSpeechToTextServiceTests.cs
■ ■■■ Tts/
■ ■■■ AzureTextToSpeechServiceTests.cs
■■■ AVA.IntegrationTests/ ← L1 integration tests (Phase 5)
■■■ UnitTest1.cs ← Placeholder, to be filled in Phase 5
```

The Conversation Pipeline (One Round-Trip)

When a user says or types something, this is the exact sequence of steps:

```
User speaks/types
■
▼
[Browser] MediaRecorder captures WebM/Opus audio
■
▼ (if voice)
[Browser] AudioContext.decodeAudioData() + OfflineAudioContext
■ Converts WebM/Opus → 16kHz 16-bit mono PCM WAV
▼
[Browser] POST /api/voicetest/transcribe (multipart/form-data)
■
▼ (server)
[AzureSpeechToTextService] WAV bytes → Azure Cognitive Services STT API
■ Returns: "I feel really anxious about my interview tomorrow"
▼
[ConversationOrchestrator.RefineTranscriptAsync]
■ Claude Haiku cleans up filler words: "um I like feel anxious" → "I feel anxious"
▼
[RuleBasedEmotionClassifier.Analyze()]
■ Keyword scan: "anxious" → EmotionType.Anxious
▼
[EfCoreContextManager.GetContextAsync()]
```

```
■ Loads last 20 messages from SQLite for this sessionId
▼
[DynamicSystemPromptBuilder.Build()]
■ Builds personalised instructions for Claude:
■ "You are AVA... User is feeling Anxious... Focus on grounding techniques..."
▼
[AnthropicLlmProvider.StreamCompletionAsync()]
■ Sends full message history + system prompt to Claude claude-sonnet-4-6
■ Streams response tokens back one-by-one
▼
[ConversationOrchestrator collects all tokens into full response string]
■
▼
[EfCoreContextManager.SaveExchangeAsync()]
■ Saves user message + AVA response to SQLite, trims to MaxHistoryLength=20
▼
[AzureTextToSpeechService.SynthesizeStreamAsync()]
■ Claude's text → Azure Neural TTS → WAV audio bytes
▼
[VoiceTestController.Chat()]
■ Returns JSON: { response, emotion, audioBase64, audioMimeType, latencyMs }
▼
[Browser] atob(audioBase64) → Uint8Array → Blob → Object URL → <audio>.play()
■
▼
User hears AVA's response in her voice ■
```

Technical Terms Explained

Networking & Communication

HTTP (HyperText Transfer Protocol)

The request-response protocol the browser uses to talk to the server. One request → one response, then the connection closes. Used for: `/chat`, `/transcribe`, `/synthesize`, `/greet` endpoints.

WebSocket

A persistent, two-way connection between browser and server. Unlike HTTP, the connection stays open and both sides can send messages at any time. Used for SignalR (real-time audio streaming). Lower latency than HTTP for continuous data.

SignalR

Microsoft's library that wraps WebSockets (with fallbacks to long-polling if WebSockets aren't available). In AVA, `VoiceHub` is a SignalR hub — clients connect to `/hub/voice` and can call server methods (`SendAudio`, `SendText`) while the server can push events back (`ReceiveAudioChunk`, `ReceiveTranscript`).

REST API

A style of HTTP API where each URL represents a resource and HTTP methods (GET, POST, DELETE) represent actions. `VoiceTestController` exposes a REST API. `[HttpPost("chat")]` means: respond to `POST /api/voicetest/chat`.

Base64

A way to encode binary data (like audio bytes) as plain ASCII text. Needed because JSON only carries text — you can't embed raw bytes in JSON. `Convert.ToBase64String(audioBytes)` on the server, `atob(base64string)` in the browser.

CORS (Cross-Origin Resource Sharing)

A browser security rule: scripts on `http://localhost:3000` cannot call APIs on `http://localhost:5163` unless the server explicitly allows it. We configured `WithOrigins("http://localhost:5163").AllowCredentials()` for SignalR. `AllowAnyOrigin()` cannot be combined with `AllowCredentials()` — this was a bug we hit and fixed.

Audio & Speech

STT (Speech-to-Text)

Converts audio recordings into text transcripts. We use **Azure Cognitive Services Speech SDK**. The service expects **16kHz, 16-bit, mono PCM WAV** audio. It returns a string like `"I feel really lonely today"`.

TTS (Text-to-Speech)

Converts text into spoken audio. We use **Azure Neural TTS** with the voice `en-US-JennyNeural` — a warm, empathetic-sounding female voice. Output format: `Riff16Khz16BitMonoPcm` (WAV). We changed from MP3 because WAV plays natively in all browsers without codec issues.

PCM (Pulse-Code Modulation)

The raw, uncompressed audio format. Every sample is a number representing the amplitude (loudness) of the sound wave at that moment. `16-bit PCM` means each sample is a number from -32768 to +32767. `16kHz` means 16,000 samples per second.

WebM/Opus

The format browsers use when recording audio with `MediaRecorder`. It's compressed (like MP3) but Azure STT cannot decode it — it only accepts raw PCM WAV. That's why the browser must convert WebM → WAV before sending audio to the server.

WAV (Waveform Audio File Format)

A container format for audio. Has a 44-byte RIFF header describing the format (sample rate, channels, bit depth) followed by raw PCM samples. We build this header manually in JavaScript's `createWavBuffer()` function.

AudioContext / OfflineAudioContext

Browser Web Audio APIs. `AudioContext` decodes compressed audio (WebM) into raw Float32 samples. `OfflineAudioContext` renders audio at a specified sample rate (16kHz) without real-time playback — perfect for format conversion. Together they resample WebM/Opus → 16kHz mono PCM.

MediaRecorder

Browser API for recording audio from the microphone. We use `onmousedown`/`onmouseup` (hold-to-record) to start/stop recording. The output is a `Blob` of WebM/Opus data.

LLM & AI

LLM (Large Language Model)

An AI model trained on vast amounts of text that can generate human-like responses. We use **Claude claude-sonnet-4-6** by Anthropic as AVA's brain.

System Prompt

A set of instructions given to the LLM before any conversation starts. It defines the AI's persona, rules, and constraints. AVA's system prompt tells Claude: "You are AVA, an empathetic emotional support agent. The user is feeling Anxious

right now. Focus on grounding techniques...". The system prompt changes dynamically based on the detected emotion.

Token

The unit of text an LLM processes. Roughly 1 token ≈ 0.75 words. Claude streams its response token by token (word by word) — we collect all tokens into a complete string before passing to TTS.

Streaming (LLM)

Instead of waiting for the entire response, the LLM sends tokens as soon as they're generated. In C#, this is modelled as `IAsyncEnumerable<string>` — an async stream of string tokens. We use `await foreach` to consume them.

IAsyncEnumerable<T>

A C# interface for an asynchronous stream of values. Like `IEnumerable<T>` but each item is produced asynchronously. Used for both LLM token streaming and TTS audio chunk streaming. Consumed with `await foreach (var item in stream)`.

Transcript Refinement

After Azure STT produces a raw transcript, a second (fast) Claude call (claude-haiku) cleans it up — removes filler words ("um", "uh", "like"), fixes repeated words, corrects mis-hearings. This gives the main LLM cleaner input. Skipped for inputs under 4 words (not worth the latency).

Emotion Detection

`RuleBasedEmotionClassifier` scans the transcript for keyword lists per emotion type. "lonely", "isolated", "nobody" → Lonely. "anxious", "nervous", "scared" → Anxious. The highest-scoring emotion wins. If no keywords match → Neutral. This is a rule-based classifier (no ML model needed — fast, deterministic, testable).

EmotionType enum

`Neutral, Sad, Anxious, Lonely, Angry, Happy, Motivated, Hopeful` — the 8 emotional states AVA can detect. Each maps to a different set of instructions in the system prompt.

Database & Persistence

EF Core (Entity Framework Core)

Microsoft's ORM (Object-Relational Mapper). Lets you write C# classes and query them with LINQ instead of writing raw SQL. EF Core generates the SQL automatically. Used in Phase 4 to store conversations in SQLite.

SQLite

A file-based relational database. No server needed — the entire database is one `.db` file on disk (`%LOCALAPPDATA%\AVA\ava_conversations.db`). Perfect for local development. Phase 5 can swap to PostgreSQL for production.

DbContext

The EF Core class that represents your database. `AvaDbContext` has two `DbSet<T>` properties: `Sessions` and `Messages` — these map to the `ConversationSessions` and `ConversationMessages` tables.

DbSet<T>

Represents a table in the database. `db.Messages.Where(m => m.SessionId == id)` generates `SELECT * FROM ConversationMessages WHERE SessionId = ...`. LINQ expressions on `DbSet` are translated to SQL.

IDbContextFactory<T>

A factory that creates new `DbContext` instances on demand. Used instead of injecting `DbContext` directly because `EfCoreContextManager` is a Singleton (app-lifetime) but `DbContext` is Scoped (request-lifetime). Injecting Scoped into Singleton causes a **captive dependency** bug. The factory is Singleton-safe.

EnsureCreated()

Creates the database schema (tables, indexes, constraints) from the EF Core model if the database file doesn't exist yet. Used in development. In production you'd use `Migrate()` with migration files that track schema changes over time.

Fluent API

A way to configure EF Core schema using method chains instead of attributes on entity classes. Done in `AvaDbContext.OnModelCreating()`. Example: `e.Property(m => m.Role).HasMaxLength(20).IsRequired()`. Keeps entity classes clean — no EF annotations on domain-ish objects.

Cascade Delete

When you delete a `ConversationSession`, EF Core automatically deletes all its `ConversationMessage` rows. Configured with `.OnDelete(DeleteBehavior.Cascade)`. Without this, deleting a session would leave orphaned message rows.

Navigation Property

A C# property on an entity that references related entities. `ConversationSession.Messages` is a navigation property — EF Core populates it when you call `.Include(s => s.Messages)`. Without `.Include()`, it's null (lazy loading is off by default).

MaxHistoryLength = 20

AVA keeps only the last 20 messages per session in context. This limits Claude's token consumption (history beyond 20 messages doesn't significantly improve response quality but increases cost). Trimming happens in `EfCoreContextManager.TrimHistoryAsync()` after every save.

Dependency Injection (DI)

Dependency Injection

Instead of a class creating its own dependencies (`new AzureSpeechToTextService()`), they are "injected" through the constructor. The DI container in `Program.cs` knows which concrete class to provide for each interface.

IServiceCollection (Program.cs)

Where all services are registered. `builder.Services.AddSingleton<IContextManager, EfCoreContextManager>()` means: "whenever something asks for `IContextManager`, give it a `EfCoreContextManager` instance."

Singleton

One instance shared across the entire application lifetime. Used for: `EfCoreContextManager`, `ConversationOrchestrator`, all Azure services. Thread-safe classes that are expensive to create.

Scoped

One instance per HTTP request. `DbContext` is scoped — each request gets a fresh context, preventing state leakage between requests.

Transient

A new instance every time it's requested. Used for lightweight, stateless services.

Options Pattern (IOptions<T>)

The safe way to read configuration settings. `ClaudeSettings` has a `SectionName = "Claude"`. In `Program.cs`: `builder.Services.Configure<ClaudeSettings>(config.GetSection("Claude"))`. In `AnthropicLlmProvider`: `IOptions<ClaudeSettings> options` — the constructor receives the typed config object. **API keys are never hardcoded** — they live in `appsettings.Development.json` which is gitignored.

Testing

xUnit

The unit testing framework used in this project. Tests are C# methods marked with `[Fact]`. The test runner (via `dotnet test`) discovers and runs all `[Fact]` methods automatically.

L0 Tests (Unit Tests)

Tests that run entirely in memory with no external dependencies (no Azure, no Claude, no disk). The "L" stands for Level. L0 = fastest, smallest scope. Every dependency is mocked or faked.

L1 Tests (Integration Tests)

Tests that test multiple components together, possibly with real databases or network calls. Phase 5 will implement these in `AVA.IntegrationTests`.

Moq

A C# mocking library. `var mockStt = new Mock<ISpeechToTextService>()` creates a fake implementation of the interface. `mockStt.Setup(s => s.TranscribeAsync(...)).ReturnsAsync("hello")` defines what the fake returns when called. `mockStt.Verify(s => s.TranscribeAsync(...), Times.Once)` asserts it was called exactly once.

FluentAssertions

A library that makes assertions read like English. Instead of `Assert.Equal(expected, actual)`, you write `actual.Should().Be(expected, "because...")`. The error messages are much clearer when tests fail.

EF Core InMemory Provider

An EF Core database provider that stores data in a .NET dictionary in RAM instead of a real database file. Used in unit tests for `EfCoreContextManager` — gives us real EF Core query behavior without disk I/O. Each test uses a unique database name (`Guid.NewGuid().ToString()`) for isolation.

IDbContextFactory (Test Helper)

We implemented `TestDbContextFactory` in the test file — a minimal `IDbContextFactory<AvaDbContext>` that creates `AvaDbContext` instances from fixed options. This is the correct pattern for testing classes that use `IDbContextFactory`.

C# Language Features Used

record

An immutable reference type with value-based equality. `public record ChatMessage(MessageRole Role, string Content)` — the constructor is auto-generated, `==` compares by value (not reference). Used for DTOs that are passed through the pipeline.

IAsyncEnumerable<T>

Async stream interface. `yield return` inside an `async` method produces items one at a time. Used for streaming TTS audio chunks and LLM tokens.

await foreach

Consuming an `IAsyncEnumerable<T>`. `await foreach (var token in _llm.StreamCompletionAsync(...))` processes each token as it arrives without blocking the thread.

CancellationToken

Passed through the entire pipeline so that if the HTTP request is cancelled (user closes the browser), all async operations can stop early. `ct.ThrowIfCancellationRequested()` is called in the TTS streaming loop.

await using

The async version of `using`. Calls `DisposeAsync()` when the block exits. Used for `DbContext` instances: `await using var db = await _factory.CreateDbContextAsync(ct)` — ensures the DB connection is released promptly.

Primary Constructor (record syntax)

`public record SynthesizeRequest(string Text, string Voice = "en-US-JennyNeural")` — the parameters become public properties automatically. Default values work just like method parameters.

What Each Phase Delivered

Phase 1 — Project Scaffolding

- Created GitHub repository
- Installed tools: .NET 8 SDK, VS Code, Git, Node.js
- Created the solution structure with `dotnet new` commands
- Set up the 4-project Clean Architecture layout
- First commit pushed to GitHub

Phase 2 — Audio Pipeline (STT + TTS)

- Created all Domain entities and enums
- Defined all Application interfaces
- Implemented `AzureSpeechToTextService` — sends WAV bytes to Azure, gets transcript
- Implemented `AzureTextToSpeechService` — sends text to Azure Jenny voice, streams WAV chunks
- Created `VoiceTestController` with `/transcribe` and `/synthesize` endpoints
- Wrote first unit tests (L0): 8 tests for STT service, TTS service, controller validation
- Connected Azure Cognitive Services credentials

Phase 3 — Full Conversation Pipeline (LLM Integration)

- Implemented `AnthropicLlmProvider` — streams Claude's response token-by-token
- Implemented `RuleBasedEmotionClassifier` — keyword-based emotion detection
- Implemented `DynamicSystemPromptBuilder` — per-emotion system prompt generation
- Implemented `InMemoryContextManager` — conversation history in RAM (ConcurrentDictionary)
- Built `ConversationOrchestrator` — the central pipeline (STT→Emotion→Context→Prompt→LLM→Save)
- Added transcript refinement (Claude Haiku cleans raw STT output)
- Built `VoiceHub` (SignalR) — real-time audio streaming over WebSocket
- Added `/chat` HTTP endpoint — full pipeline, returns JSON with base64 WAV audio

- Added auto-greeting: `/greet` endpoint synthesizes AVA's opening message on page load
- Built `wwwroot/index.html` — browser UI with hold-to-record mic, text input, audio player
- Added WebM→WAV conversion in browser (AudioContext + OfflineAudioContext)
- Fixed audio playback: switched from SignalR binary streaming to HTTP JSON + base64
- Fixed HTTP header bug: Claude responses contain em-dashes (non-ASCII) — moved all response data to JSON body
- Grew test suite to 10 unit tests

Phase 4 — Persistent Storage (SQLite + EF Core)

- Added EF Core + SQLite packages to Infrastructure project
- Added EF Core Design package to Api project (enables migration tooling)
- Added EF Core InMemory package to UnitTests project (for isolated tests)
- Created `ConversationSession` EF Core entity (one DB row per browser session)
- Created `ConversationMessage` EF Core entity (one DB row per message, with role + emotion)
- Created `AvaDbContext` with Fluent API schema config, cascade deletes, and query indexes
- Implemented `EfCoreContextManager` using `IDbContextFactory` pattern (Singleton-safe)
- Replaced `InMemoryContextManager` with `EfCoreContextManager` in DI registration
- Fixed `ClearSessionAsync` — EF Core InMemory provider ignores cascade deletes; messages must be explicitly deleted before the session row
- Added `localStorage` session ID persistence in browser — same `sessionId` survives page refreshes so EF Core loads the right history
- Added **"+ New Conversation"** button to clear `localStorage` and start fresh
- Added `ExtractNameFromText()` in `ConversationOrchestrator` — detects "Hi this is Akhil", "my name is Akhil" etc. and passes name to context manager
- Added cross-session memory lookup in `EfCoreContextManager` — when a session has no messages but a name is known, finds the most recent prior session with that name and copies its history
- Updated `DynamicSystemPromptBuilder` — added `YOUR MEMORY` section explicitly telling Claude it has persistent memory and must never say "I don't remember you"
- Updated `BuildUserContext` — tells Claude exactly how many messages of history it has loaded, so it responds confidently
- Wrote 7 new unit tests for `EfCoreContextManager`
- **Total unit tests: 38**

Phase 5 — Integration Tests + CI/CD + Migrations

- Set up `AVA.IntegrationTests` project with `Microsoft.AspNetCore.Mvc.Testing`, EF Core InMemory, Moq, FluentAssertions
- Added `public partial class Program {}` to `Program.cs` — required for `WebApplicationFactory<Program>` to work
- Wrote **7 pipeline integration tests** (`ConversationPipelineTests.cs`) — real `ConversationOrchestrator` + real `EfCoreContextManager` + real emotion classifier + real prompt builder + mocked LLM + mocked Azure. Tests cover: first message saves to DB, second message includes prior context, emotion detection, empty LLM fallback, session isolation, history persistence, system prompt contains emotion guidance
- Wrote **8 HTTP endpoint integration tests** (`ChatEndpointTests.cs`) — boots the full ASP.NET Core app in-memory using `WebApplicationFactory`, sends real HTTP requests, asserts real JSON responses for `/chat`, `/greet`, `/ping`, `/health`
- Created **GitHub Actions CI** (`.github/workflows/ci.yml`) — triggers on every push/PR to `main`/`develop`, runs `dotnet restore` → `dotnet build --configuration Release` → `dotnet test`, uploads test results + coverage artifacts
- Created **EF Core initial migration** (`20240601000000_InitialCreate.cs`) + model snapshot — versioned schema management

- Replaced `EnsureCreatedAsync()` with `MigrateAsync()` in `Program.cs`
- Fixed `MigrateAsync` crash in tests — InMemory provider doesn't support migrations; added `if (db.Database.IsRelational())` check to use `MigrateAsync` for SQLite and `EnsureCreatedAsync` for InMemory
- Fixed pipeline test mock interfe